

Projekat iz predmeta Dizajn kompajlera

Problem

Potrebno je napraviti interpreter za jednostavni programski jezik. Za izradu zadataka koristiti flex, bison, kao i metod evaluacije programa pomoću abstraktnih sintaksnih stabla, kao na posljednjim laboratorijskim vježbama.

Finalni jezik, nakon svakog implementiranog zadatka, treba da podržava sljedeće:

- jedini aritmeticki tip podatka je *integer*,
- aritmetičke, relacione i logičke operacije nad integerima,
- definiciju varijabli tipa integer, sa varijablama imenovanim kao u C jeziku,
- blok koda pomoću `{ i }`,
- kontrolu toka pomoću *if-then-else* naredbi,
- petlje pomoću *while-do* naredbe,
- petlje pomoću *for* naredbe,
- definiciju i pozivanje funkcija.

U prilogu zadatke se nalazi šablon na osnovu kojeg možete nastaviti dalje izradu zadatke.

Studenti treba da odrade problem u opsegu koliko je moguće. U spomenutom šablonu se nalazi Makefile koji kompajlira program, ali pored toga i podržava komandu `make test` koja će testirati ulaze u program i reći uolikoj mjeri je problem implementiran.

Zadaću predajete koristeći Google classroom kao jedan file, sa imenom `ime_prezime_zadaca7.tar.gz`. Otpakovana arhiva treba dati folder imena `ime_prezime_zadaca7` unutar koje se nalazi projekat.

Početna gramatika

Kod u prilogu sadrži evaluator sa sljedećom gramatikom:

```
program = program statement
        | epsilon
        ;
```

```
statement = expression ';' ;
```

```

;

expression = expression + expression
            | expression - expression
            | NUM
;

```

Evaluacija svakog *statementa* se izvršava u glavnom pravilu za program, koji vrti programsku petlju. Za razliku od primjera na vježbama, ovaj evaluator prikazuje ispis korisniku samo kad korisnik unese `print` komandu.

U semantičkim akcijama se alociraju i uvezuju čvorovi, koji čine stablo za izvršenje. Čvorovi su definirani u fileu `expression_tree.hpp`.

Posebna struktura podataka predstavlja environment, koji će čuvati tabelu simbola, te ostale specifične varijable koje smatrate da su potrebne. Svaka semantička akcija će imati pristup objektu tipa `Environment`, te će svaki čvor prilikom evaluacije imati pristup tom objektu.

Zadatak 0 - implementirati print funkcionalnost

Program treba moći isprintati neku vrijednost pomoću `print` naredbe i `PrintNode` čvora. Za razliku od primjera sa vježbi, `statement` se evaluira, dok će `print node`, interno kada je evaluiran pozvati `printf`.

```

print(42);
> 42

```

Zadatak 1 - implementirati aritmetičke i logičke operacije koje nedostaju

Trenutni kod podržava sabiranje i oduzimanje. Potrebno je implementirati ostale operatore:

- aritmetičke sa znakovima `+`, `-`, `*`, `/`,
- unarni minus,
- relacije (`<`, `<=`, `>`, `>=`), operator poređenja (`=`) i različito (`<>`),

Postoji mogućnost korištenja zagrada kako bi precizno odredili redoslijed izvršenja navedenih operacija.

Napomena: Operator dodijele će biti `:=`.

Vaš zadatak:

- implementirati tokene potrebne za ove operacije, za svaku operaciju dodati odgovarajuće pravilo u gramatiku,

- Za svaku operaciju napraviti novi tip čvora u **expression_tree** arhitekturi, po uzoru na postojeće čvorove za sabiranje i oduzimanje,
- U semantičkoj akciji za ova pravila alocirati i uvezati novi čvor.

Konačni cilj je da program može izvršavati kod tipa:

```
print(1 = 1);
> 1
print(1 <> 1);
> 0
print(1 < 5);
> 1
print(1 > 5);
> 0
print(10 / 5);
> 2
print(2 * 5);
> 10
print(5 - 4 * 3);
> -7
print(4 * 4 * 4 + 3 - 2 + 3);
> 68
print(1 * 2 + 1);
> 3
print(2 - 10 + 15);
> 7
print(2 + 2 * 2);
> 6
print --5;
> 5
print (5 + 10 * (100 - 25))
print (5 * 10 or 100 * 25)
```

Zadatak 2 - varijable, dodjela i čitanje vrijednosti

Sve varijable treba da žive u klasi **Environment**, koja je definirana u fileu **environment.hpp**. Potrebno je u klasu dodati odgovarajući kontejner koji bi čuvao vrijednosti koje su trenutno asocirane sa imenom varijable.

Imena slijede pravila C jezika za imenovanje varijabli. Novi tip tokena **ID** čuva semantičku vrijednost niza karaktera, koji treba dodati u **union**.

Potrebno je dodati dva nova pravila u nizu alternativa za **expression**: - pisanje **ID** **':'** **expression**, - čitanje **ID**.

Obratiti pažnju da se dodjela vrijednosti i čitanje dešavaju tek pri evaluaciji statementa, tako da i za ovu namjenu trebate implementirati dva nova čvora, sa

imenom `AssignmentNode` i `VariableNode`.

Vaš zadatak:

- Dodati novi token tipa `ID`,
- Dodati novi operator `:=`,
- Dodati nova pravila u expression alternative,
- Napraviti novi tip podatka `AssignmentNode` i alocirati ga u semantičkoj akciji dodjele,
- Napraviti novi tip podatka `VariableNode` i alocirati ga u semantičkoj akciji dohvaćanja vrijednosti,

Konačni cilj je da program može izvršavati kod tipa:

```
a := 10;
print(a);
> 10
b := a + 10;
print(b);
> 20
print(c);
> 0
a := b := c := 20 - 5 + 5;
print(a);
> 20
print(b);
> 20
print(c);
> 20
```

Zadatak 3 - if else izraz

Potrebno je dodati produkcijska pravila i čvorove za `if-then` i `if-then-else` izraze. Njihova produkcijska pravila će biti smještena kao `statement`. Obratite pažnju na `shift reduce` konflikte, te napravite ugniježdeni izrazi rade pravilno.

Vaš zadatak:

- Dodati pravila u statement alternative,
- U semantičkoj akciji pravilo alocirati čvor,
- Implementirati čvorove i evaluaciju.

Cilj je da program može izvršavati `if` i `else` naredbe, u sljedećem formatu:

```
if 1 then print(10);
> 10
if 0 then print(10);
if 1 then print(10); else print(20);
> 10
```

```
if 0 then print(10); else print(20);
> 20
```

Zadatak 4 - while petlja

Sintaksa za while petlju ima sljedeću strukturu:

```
statement : WHILE expression DO statement
```

Vaš zadatak:

- Dodati pravilo u statement alternative,
- U semantičkoj akciji za to pravilo alocirati čvor tipa `WhileNode`.
- Implementirati `evaluate` metod za `WhileNode`, na način da dok se god uslov evaluira na tačno, izvrši se tijelo petlje

Cilj je da se sljedeći kod uspješno izvrši:

```
a = 4;
while a do a = a - 1;
print(a);
> 0
```

Zadatak 5 - blok koda

Potrebno je implementirati novi tip čvora koji modelira blok koda. Rezultat bloka koda je jedan čvor koji u sebi čuva niz izraza koji su u tijelu bloka koda, te pozivom `evaluate`, evaluiraju se redom izrazi iz niza.

Sintaksa bi bila sljedeća:

```
block_code: { statement_list }
```

```
statement_list : statement_list statement
                | epsilon
```

Za semantičku vrijednost `statement_list` pravila bi trebali koristiti novi tip podatka u `union`-u, koji bi bio pokazivac na vektor `ExpressionNode`-ova. Posto je u pitanju dinamički alociran vektor, `std::vector<ExpressionNode>`, sam vektor možemo alocirati u `epsilon` pravilu, a potom za svaki `statement` u rekurzivnom pravilu ubaciti `Node` u vektor. Drugo rješenje bi bilo da alociramo vektor u posebnom markeru.

Primjer:

```
a := 10;
{
    b := 10;
    print(a + b);
}
```

```
> 30
}
```

Zadatak 6 - if/else i while petlja sa blokom koda

Iako smo napraviti if/else i while petlju da rade sa jednim izrazom, ako ste dodali blok koda kao **izraz/statement** omogućili ste i blok koda kao tijelo if-then-else i while izraza.

```
if 5 then
{
    x = 10;
}
else
{
    x = 20;
}
print(x);
> 10

a = 5;
while a do
{
    a = a - 1;
    print(a);
}
```

Zadatak 7 - definicija i poziv funkcije

Sintaksa definicije funkcije bi bila sljedeća:

```
statement : FN ID '(' id_list ')' block_code;
```

dok bi za pozive funkcija imali sljedeću sintaksu:

```
expression : ID '(' expression_list ')'
```

Potrebno je implementirati 3 tipa čvora koji su namjenjeni za pozive funkcija:

- **FunctionNode** koji predstavlja samu funkciju i njeno tijelo. Evaluacija ovog čvora je zapravo evaluacija same funkcije. U **Environment** varijabli potrebno je napraviti mapu funkcija.
- **FunctionDefinitionNode** je čvor koji predstavlja definiciju funkcije. Njegova uloga je da kreira **FunctionNode** i postavi ga u mapu funkcija koja se nalazi u **Environment** varijabli, čineći ga time dostupnim za pozive.
- **FunctionCallNode** je čvor koji predstavlja poziv funkcije. Njegova uloga je dohvaćanje **FunctionNode**-a iz mape unutar **Environmenta**, evaluacija

izraza koji su parametri pozivu funkcije, i pripremanje tih parametara za tijelo funkcije.

- Metod `evaluate` za `FunctionCallNode` treba da provjeri da li postoji funkcija sa datim imenom i dohvati pokazivač na funkciju. U tom momentu, dostupni su vektor `ExpressionNode`-ova koji predstavlja parametre pozivu funkcije, i vektor imena argumenata koje funkcija očekuje. Ovaj metod treba da evaluiira `ExpressionNode`-ove i njihov rezultat spremi u mapu varijabli pod imenom koje je parametar funkcije. Na primjer ako imamo funkciju `foo(a,b,c)` i pozovemo je na način `foo(1, 2, 3)`, potrebno je u environment varijable ubaciti key-value parove `{a, 1}`, `{b,2}`, `{c, 3}`.
- Na kraju treba evaluirati samu funkciju.

Nakon ovog dijela, sljedeći kod bi trebao da se može izvršiti

```
fn foo() {
  print(1);
  print(2);
  print(3);
}

foo()
>1
>2
>3
foo()
>1
>2
>3

test := 10;
bar(a, b, c) {
  print(a);
  print(b);
  print(c);
}

bar(1, 2 + 2 , test)
>1
>4
>10
```

Zadatak 8 - Povratna vrijednost funkcije

Potrebno je osmisliti i implementirati način da se iz funkcije vrati vrijednost natrag u pozivajuću funkciju. U sintaksu programa bi ubacili pravilo:

```
RETURN expression;
```

Predaja zadaće

Svaki pokušaj prepisivanja i plagijata će biti oštro sankcionisan od strane predmetnog asistenta. Porebno je predati zadaću na google classroom u arhivi sa imenom `ime_prezime_projekat.tar.gz`. Ne predavati binarne fileove. Otpakovana tar arhiva treba dati folder sa imenom `ime_prezime_projekat`.